AMB W SPC - Advanced Manufacturing Business with Statistical Process Control

License MIT
Frappe v15.0+
ERPNext v15.0+
Python 3.8+

🎯 Overview

AMB W SPC is a comprehensive Frappe/ERPNext application designed for modern manufacturing environments, providing advanced Statistical Process Control (SPC), Shop Floor Control (SFC), and real-time manufacturing operations management.

🏭 Key Features

- 📊 **Statistical Process Control (SPC)**
- Real-time control charts and process monitoring
- Automated alert generation and escalation
- Process capability analysis (Cp, Cpk calculations)
- Trend analysis and predictive insights
- 🏗️ **Shop Floor Control (SFC)**
- Real-time production monitoring
- Work order routing and operations tracking
- Equipment status monitoring
- Operator management and skill tracking
- 📡 **Real-Time Data Integration**

- Sensor data collection and processing
- IoT device integration
- Live dashboard updates
- Automated data archiving
- 👷 **Operator Management**
- Skill-based work assignments
- Attendance tracking
- Performance monitoring
- Training management
- 🔍 **Quality Management**
- FDA compliance tools
- Deviation management (CAPA)
- Electronic signatures
- Audit trail maintenance
- ⚙️ **Manufacturing Operations**
- Station-based OEE calculations
- Production scheduling
- Material tracking
- Maintenance management

🚀 Quick Start

Prerequisites

- **Frappe Framework**: v15.0 or higher
- **ERPNext**: v15.0 or higher
- **Python**: 3.8 or higher
- **Node.js**: 16 or higher

- **MariaDB**: 10.5 or higher

📦 Installation

Method 1: Using Frappe Bench (Recommended)

```
# 1. Navigate to your frappe-bench directory
cd /path/to/frappe-bench

# 2. Get the app from GitHub
bench get-app https://github.com/your-username/amb_w_spc.git

# 3. Install the app on your site
bench --site your-site-name install-app amb_w_spc

# 4. Restart bench
bench restart
```

Method 2: Manual Installation

```
# 1. Clone the repository
git clone https://github.com/your-username/amb_w_spc.git
cd amb_w_spc

# 2. Install dependencies
pip install -r requirements.txt

# 3. Copy to your Frappe apps directory
cp -r . /path/to/frappe-bench/apps/amb_w_spc

# 4. Install using bench
cd /path/to/frappe-bench
bench --site your-site-name install-app amb_w_spc
```

Method 3: Production Deployment

```
# Download the latest release
wget https://github.com/your-username/amb_w_spc/releases/latest/
download/amb_w_spc_production.tar.gz

# Extract and install
tar -xzf amb_w_spc_production.tar.gz
cd amb_w_spc
bash install.sh
```

⚡ Quick Setup

After installation, run the setup wizard:

```
# Initialize the system
bench --site your-site-name execute
amb_w_spc.system_integration.installation.install_spc_system.setup_system

# Create sample data (optional)
bench --site your-site-name execute
amb_w_spc.fixtures.create_sample_data
```

📖 Documentation

🏗️ Architecture

The application is organized into the following modules:

Module	Purpose	Key Features
Core SPC	Statistical Process Control engine	Control charts, alerts, data analysis
Manufacturing Operations	Production management	Work orders, routing, scheduling
Shop Floor Control	Real-time floor management	Station monitoring, equipment tracking
Operator Management	Workforce management	Skills, attendance, assignments
Sensor Management	IoT integration	Data collection, sensor configuration
Real Time Monitoring	Live dashboards	Alerts, notifications, status updates
Quality Management	Quality assurance	CAPA, deviations, compliance
FDA Compliance	Regulatory compliance	Audit trails, electronic signatures
Plant Equipment	Equipment management	Maintenance, calibration, configuration
System Integration	Integration tools	APIs, workflows, permissions

🔧 Configuration

Initial Setup

1. **Configure Manufacturing Stations**
   `Manufacturing > Setup > Manufacturing Station`
2. **Setup SPC Parameters**
   `Quality > SPC Setup > SPC Parameter Master`
3. **Configure Sensors** (if using IoT)
   `Manufacturing > Sensor Management > Sensor Configuration`
4. **Setup Operators**
   `Manufacturing > Operator Management > SFC Operator`

Advanced Configuration

- **Scheduler Settings**: Customize background job frequencies in `hooks.py`
- **Alert Thresholds**: Configure in SPC Parameter Control Limits
- **Workflow Customization**: Modify in System Integration > Workflows

📊 Usage Examples

Creating SPC Control Charts

```
# Example: Create a control chart for a process parameter
frappe.get_doc({
    "doctype": "SPC Control Chart",
    "parameter": "Temperature",
    "chart_type": "X-Bar R",
    "upper_control_limit": 75.0,
    "lower_control_limit": 65.0,
    "target_value": 70.0
}).insert()
```

Real-time Data Collection

```
# Example: Log sensor data
frappe.get_doc({
    "doctype": "SPC Data Point",
    "parameter": "Temperature",
    "value": 72.5,
    "timestamp": frappe.utils.now(),
    "station": "Station-001"
}).insert()
```

🛠️ Development

Setting up Development Environment

```
# 1. Fork and clone the repository
git clone https://github.com/your-username/amb_w_spc.git
cd amb_w_spc

# 2. Create virtual environment
python -m venv venv
source venv/bin/activate  # Linux/Mac
# venv\Scripts\activate  # Windows

# 3. Install development dependencies
pip install -r requirements.txt
pip install -r requirements-dev.txt

# 4. Install pre-commit hooks
pre-commit install
```

Running Tests

```
# Run all tests
bench --site test_site run-tests --app amb_w_spc

# Run specific test
bench --site test_site run-tests --app amb_w_spc --module
amb_w_spc.tests.test_spc

# Run with coverage
bench --site test_site run-tests --app amb_w_spc --coverage
```

Building for Production

```
# Create production build
python setup.py sdist bdist_wheel

# Create distribution package
bash create_distribution.sh
```

🔧 API Reference

REST API Endpoints

Endpoint	Method	Description
`/api/resource/SPC Data Point`	GET/POST	Manage SPC data points
`/api/resource/SFC Operator`	GET/POST	Operator management
`/api/resource/Manufacturing Station`	GET/POST	Station configuration
`/api/method/amb_w_spc.api.get_live_data`	GET	Real-time dashboard data
`/api/method/amb_w_spc.api.trigger_alert`	POST	Manual alert triggering

WebSocket Events

- `sensor_data_update`: Real-time sensor data
- `station_status_change`: Manufacturing station updates
- `new_alert`: SPC alert notifications
- `operator_checkin`: Operator status changes

🔒 Security

- All API endpoints require authentication
- Role-based access control (RBAC)
- Audit trail for all transactions
- Electronic signature support for FDA compliance
- Data encryption for sensitive information

📈 Performance

- **Database Optimization**: Indexed queries for real-time performance
- **Caching**: Redis-based caching for frequently accessed data
- **Background Jobs**: Async processing for heavy operations
- **Data Archiving**: Automated cleanup of historical data

🤝 Contributing

We welcome contributions! Please see our Contributing Guidelines for details.

📋 Development Guidelines

1. **Code Style**: Follow PEP 8 for Python, Frappe conventions for JavaScript
2. **Testing**: Write tests for all new features
3. **Documentation**: Update documentation for any changes
4. **Commit Messages**: Use conventional commit format

🐛 Bug Reports

Please use the GitHub Issues page to report bugs.

📄 License

This project is licensed under the MIT License - see the LICENSE file for details.

🆘 Support

- **Documentation**: Wiki
- **Community**: Discussions
- **Issues**: GitHub Issues
- **Email**: support@ambsystems.com

🎉 Acknowledgments

- Built on the excellent Frappe Framework
- Inspired by modern manufacturing best practices
- Thanks to all contributors and the open-source community

Made with ❤️ for the Manufacturing Community

📊 Statistics

- **15 Modules**: Comprehensive manufacturing coverage
- **49 DocTypes**: Extensive data model
- **132 Python Files**: Robust backend logic
- **Real-time Processing**: Live data updates
- **FDA Compliant**: Regulatory ready